a `std::future` with a task; you can also do it by wrapping the task in an instance of the `std::packaged_task<>` class template or by writing code to explicitly set the values using the `std::promise<>` class template. `std::packaged_task` is a higher-level abstraction than `std::promise`, so I'll start with that.

4.2.2. Associating a task with a future

`std::packaged_task<>` ties a future to a function or callable object. When the `std::packaged_task<>` object is invoked, it calls the associated function or callable object and makes the future *ready*, with the return value stored as the associated data. This can be used as a building block for thread pools (see chapter 9) or other task management schemes, such as running each task on its own thread, or running them all sequentially on a particular background thread. If a large operation can be divided into self-contained sub-tasks, each of these can be wrapped in a `std::packaged_task<>` instance, and then that instance passed to the task scheduler or thread pool. This abstracts out the details of the tasks; the scheduler just deals with `std::packaged_task<>` instances rather than individual functions.

The template parameter for the `std::packaged_task<>` class template is a function signature, like `void()` for a function taking no parameters with no return value, or `int(std::string&,double*)` for a function that takes a non-`const` reference to a `std::string` and a pointer to a `double` and returns an `int`. When you construct an instance of `std::packaged_task`, you must pass in a function or callable object that can accept the specified parameters and that returns a type convertible to the specified return type. The types don't have to match exactly; you can construct a `std::packaged_task<double(double)>` from a function that takes an `int` and returns a `float` because the types are implicitly convertible.

The return type of the specified function signature identifies the type of the `std::future<>` returned from the `get_future()` member function, whereas the argument list of the function signature is used to specify the signature of the packaged task's function call operator. For example, a partial class definition for `std::packaged_task<std::string(std::vector<char>*,int)>` would be as shown in the following listing.

Listing 4.8. Partial class definition for a specialization of `std::packaged_task< >`

```
template<>
class packaged_task<std::string(std::vector<char>*,int)>
{
public:
    template<typename Callable>
    explicit packaged_task(Callable&& f);
    std::future<std::string> get_future();
    void operator()(std::vector<char>*,int);
};
```

The `std::packaged_task` object is thus a callable object, and it can be wrapped in a `std::function` object, passed to a `std::thread` as the thread function, passed to another function that requires a callable object, or even invoked directly. When the `std::packaged_task` is invoked as a function object, the arguments supplied to the function call operator are passed on to the contained function, and the return value is stored as the